Gjett tallet

Tilfeldige heltall – ett spill steg for steg og to oppgaver

Navn: _____ Klasse: _____

Slik bruker du heftet

I hefte 2 fikk CPU-en velge mellom tre ting. I dette heftet lærer du å lage et tilfeldig **heltall**, for eksempel fra 1 til 100 eller fra 1 til 6. Med det kan du lage mange nye spill. Først bygger du spillet **Gjett tallet** steg for steg. Etterpå lager du **Terningduell** og **Gangetabell-trener** på egen hånd.

Du bruker det du kan fra før: variabler utenfor funksjonene, knapper, input, `Number()`, `textContent`, `${ }`, `if` / `else if` / `else` og `++`. Skriv av koden selv, lagre, og **test etter hvert steg**.

Nytt i dette heftet: tilfeldig heltall med `Math.floor`

`Math.random()` gir et tall som `0.4718`. Det er ikke et tall vi kan bruke i et spill med terninger. Vi gjør det om til et heltall i tre steg:

Steg	Kode	Eksempel
1. Lag et tilfeldig tall	`Math.random()`	`0.4718`
2. Gang med 100	`Math.random() * 100`	`47.18`
3. Kutt bort desimalene	`Math.floor( ... )`	`47`
4. Legg til 1	`Math.floor(Math.random() * 100) + 1`	`48`

`Math.floor` kutter bort alt etter punktum. Den runder alltid **ned**, også `47.99` blir `47`.

Forutsi: Fyll ut resten av tabellen. Den første raden er gjort for deg.

`Math.random()` gir	`* 100`	`Math.floor( ... )`	`+ 1`
`0.4718`	`47.18`	`47`	`48`
`0.0312`			
`0.9999`			
`0.0`			

Hva er det minste og det største tallet du kan få?

Svar: _____

> **Oppskriften – lær den utenat:** Et tilfeldig heltall fra 1 til N lager du med `Math.floor(Math.random() * N) + 1`. En terning (1 til 6) blir `Math.floor(Math.random() * 6) + 1`.

Gjett tallet

CPU-en tenker på et tall fra 1 til 100. Du gjetter, og CPU-en svarer «for høyt» eller «for lavt» til du har funnet det. Lag filen `gjett-tallet.html`.

Slik skal spillet se ut

```
+-------------------------------------------------+
|                   Gjett tallet                  |
|    Jeg tenker på et tall mellom 1 og 100.       |
|               [  37  ]   [Gjett]                |
|          37 er for lavt. Prøv høyere!           |
|                Antall forsøk: 3                 |
|                  [Nytt spill]                   |
+-------------------------------------------------+
```

Steg 1 – Skjelettet og CSS-en

Skriv `!` og trykk Tab. Endre `<title>` til `Gjett tallet`, og skriv denne CSS-en i et `<style>`-element rett over `</head>`:

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background-color: #f0f4f8;
}
input {
  font-size: 20px;
  padding: 10px;
  width: 100px;
  text-align: center;
  border: 2px solid #1f3a5c;
  border-radius: 10px;
}
```

```
button {
  font-size: 20px;
  padding: 10px 20px;
  border: none;
  border-radius: 10px;
  background-color: #1f3a5c;
  color: white;
}
button:hover {
  background-color: #3d6a9e;
}
#txtHint {
  font-size: 24px;
  font-weight: bold;
}
```

Steg 2 – HTML-en

Skriv dette inni `<body>`:

```
  <h1>Gjett tallet</h1>
  <p>Jeg tenker på et tall mellom 1 og 100. Klarer du å gjette det?</p>

  <input type="number" id="inpGjett">
  <button onclick="gjett()">Gjett</button>

  <p id="txtHint"></p>
  <p id="txtForsok"></p>
  <button onclick="nyttSpill()">Nytt spill</button>
```

Lagre og se på siden. Hvilke av elementene skal JavaScript **lese fra**, og hvilke skal JavaScript **skrive til**?

Svar: _____

Steg 3 – Det hemmelige tallet

Skriv en `<script>`-blokk nederst i `<body>`, rett over `</body>`:

```
<script>
  let hemmelig = Math.floor(Math.random() * 100) + 1;

  // JUKSELINJE – bare for å teste. Slettes i steg 6!
  txtHint.textContent = `Psst! Tallet er ${hemmelig}`;
</script>
```

`hemmelig` står **utenfor** funksjonene, fordi spillet skal huske tallet mellom hver gang du gjetter. Jukselinjen står heller ikke i en funksjon. Derfor kjører den med én gang siden lastes.

Oppdater siden (F5) noen ganger. Får du det samme tallet hver gang? Hvorfor / hvorfor ikke?

Svar: _____

Steg 4 – Gjett!

Legg til funksjonen `gjett` under jukselinjen:

```
function gjett() {
  let tall = Number(inpGjett.value);

  if (tall == hemmelig) {
    txtHint.textContent = `Riktig! Tallet var ${hemmelig}.`;
  } else if (tall < hemmelig) {
    txtHint.textContent = `${tall} er for lavt. Prøv høyere!`;
  } else {
    txtHint.textContent = `${tall} er for høyt. Prøv lavere!`;
  }
}
```

Dette er **Hent – Regn – Vis** igjen: Vi henter tallet du skrev, sammenligner det med det hemmelige tallet, og viser et hint.

Forutsi: Det hemmelige tallet er `42`. Hva står det på siden når du gjetter disse tallene?

Du gjetter	Hva står på siden?
`50`	
`42`	
`7`	

Test spillet. Bruk jukselinjen, så vet du hva som er riktig svar.

Steg 5 – Tell forsøkene

Spillet skal telle hvor mange ganger du har gjettet, akkurat som telleren i hefte 1. Legg til en ny variabel under `hemmelig`:

```
let forsok = 0;
```

Legg så til disse to linjene i `gjett`, rett **under** `let tall = ...`:

```
  forsok++;
  txtForsok.textContent = `Antall forsøk: ${forsok}`;
```

Endre også teksten når du gjetter riktig til: `` `Riktig! Du brukte ${forsok} forsøk.` ``

Undersøk: Hvorfor må `let forsok = 0;` stå utenfor funksjonen? Flytt linjen inn i `gjett`, rett over `forsok++;`, og test. Hva skjer? Flytt den tilbake etterpå.

Svar: _____

Steg 6 – Nytt spill

Nå skal knappen «Nytt spill» trekke et nytt hemmelig tall og nullstille alt. **Slett jukselinjen og kommentaren over den.** Legg så til denne funksjonen nederst i `<script>`:

```
function nyttSpill() {
  hemmelig = Math.floor(Math.random() * 100) + 1;
  forsok = 0;
  txtHint.textContent = "";
  txtForsok.textContent = "";
  inpGjett.value = "";
}
```

- Vi skriver ikke `let` her. Variablene finnes allerede – vi gir dem bare nye verdier.
- `inpGjett.value = "";` tømmer input-feltet. Før har du **lest** `.value`. Nå **skriver** du til den.

Steg 7 – Spill og bli bedre

Spill noen runder uten juks. Hvilket tall er lurest å gjette **først**? Hva gjetter du hvis svaret var «for lavt»? Hvor mange forsøk trenger du på det meste med den beste strategien?

Svar: _____

Utvidelser til Gjett tallet

★ **Maks 10 forsøk.** Når `forsok` blir 10 uten riktig svar, skal det stå «Du tapte! Tallet var ...».

★ **Rekord.** Lag `let rekord = 1000;` utenfor funksjonene. Når du gjetter riktig og `forsok` er mindre enn `rekord`, blir `forsok` den nye rekorden. Vis rekorden på siden.

★★ **Vanskelighetsgrad.** Lag knappene Lett (1–10), Middels (1–100) og Vanskelig (1–1000). Hint: Lag en variabel `maks` og bruk den i oppskriften: `Math.floor(Math.random() * maks) + 1`.

★★ **Nesten!** Hvis du er mindre enn 5 unna, skal det stå «Veldig nær!». Hint: `tall > hemmelig - 5 && tall < hemmelig + 5`

★★★ **Bytt roller.** Du tenker på et tall, og CPU-en gjetter. Du svarer med knappene «For høyt», «For lavt» og «Riktig». Hint: Husk minste og største mulige tall i to variabler.

Oppgave A: Terningduell

Du og CPU-en kaster hver deres terning. Den som får høyest, får ett poeng. Den første som får 10 poeng, vinner kampen. Nå får du ingen ferdig kode – men du kan se på Gjett tallet og stein, saks, papir. Lag filen `oppgave-a-terningduell.html`.

Slik skal det se ut

```
+-------------------------------------------------+
|                   Terningduell                  |
|                     [Kast!]                     |
|            Du kastet 5. CPU kastet 3.           |
|              Du vant denne runden!              |
|                   Du 4 – 2 CPU                  |
+-------------------------------------------------+
```

Krav – kryss av når du er ferdig

- [ ] Knappen «Kast!» kjører en funksjon.
- [ ] Du og CPU-en får hvert deres tilfeldige tall fra 1 til 6.
- [ ] Den som kaster høyest, får ett poeng. Likt gir ingen poeng.
- [ ] Siden viser begge kastene, hvem som vant runden og stillingen.
- [ ] Når en av dere har 10 poeng, står det hvem som vant kampen.
- [ ] Siden har CSS som gjør den pen og lett å lese.

Hjelp til å komme i gang

- Planlegg før du koder. Fyll ut:
  - Hent: _____
  - Regn: _____
  - Vis: _____
- Poengene må huskes mellom hvert kast. Hvor må `let duPoeng = 0;` og `let cpuPoeng = 0;` stå?
- Et kast lager du slik: `let duKast = Math.floor(Math.random() * 6) + 1;`
- Bruk `if`, `else if` og `else` for å finne ut hvem som vant runden.
- Sjekk om noen har vunnet kampen **etter** at poengene er lagt til.

Test smart. Bruk jukselinjer som i hefte 2, for eksempel `duKast = 6;` og `cpuKast = 2;` rett etter kastene. Fyll ut tabellen, og slett jukselinjene etterpå.

Du kaster	CPU kaster	Hvem får poeng?
6	2	
3	3	
1	4	

Ekstra utfordring

- Lag en knapp «Ny kamp» som setter poengene til 0 og tømmer teksten.
- Vis terningsymboler i stedet for tall: ⚀ ⚁ ⚂ ⚃ ⚄ ⚅. Hint: en `if`-kjede.
- Kast to terninger hver, og bruk summen (2–12). Dobbel seks gir 3 poeng!
- Gjør det umulig å fortsette å kaste når kampen er vunnet.

Oppgave B: Gangetabell-trener

Lag et program som gir deg tilfeldige gangestykker fra den lille gangetabellen. Du skriver svaret, og programmet sjekker om det er riktig. Lag filen `oppgave-b-gangetabell.html`.

Slik skal det se ut

```
+------------------------------------------------+
|                Gangetabell-trener              |
|                    7 x 8 = ?                   |
|               [  56  ]   [Sjekk]               |
|                  [Nytt stykke]                 |
|                Riktig! 7 x 8 = 56              |
|          Poeng: 5     Riktige på rad: 3        |
+------------------------------------------------+
```

Krav – kryss av når du er ferdig

- [ ] Knappen «Nytt stykke» lager to tilfeldige tall fra 1 til 10 og viser stykket.
- [ ] Du skriver svaret i et input-felt og trykker «Sjekk».
- [ ] Riktig svar: Det står «Riktig!», og både poengene og antall riktige på rad øker med 1.
- [ ] Feil svar: Det står hva det riktige svaret var, og antall riktige på rad blir 0.
- [ ] Poengene og antall riktige på rad vises hele tiden.

Hjelp til å komme i gang

- Du trenger **to** funksjoner: `nyttStykke()` og `sjekk()`.
- Begge funksjonene må vite hvilke tall som er i stykket. Derfor må `let tallA = 0;` og `let tallB = 0;` stå **utenfor** funksjonene. I `nyttStykke` gir du dem nye verdier – uten `let`!
- Skriv `x` i teksten på siden, men `*` i koden: `` `${tallA} x ${tallB} = ?` `` og `tallA * tallB`.
- I `sjekk` henter du svaret med `Number( ... )` og sammenligner med `if (svar == tallA * tallB)`.

Test programmet. Svar riktig og feil med vilje, og fyll ut tabellen.

Stykke	Ditt svar	Hva står på siden?	Poeng	På rad
	riktig			
	riktig			
	feil			
	riktig			

Ekstra utfordring

- Tøm input-feltet og vis et nytt stykke automatisk når svaret er riktig. Hint: En funksjon kan kalle en annen.
- Husk den lengste rekken av riktige svar som en rekord.
- Lag et input-felt der du velger hvilken gangetabell du vil øve på, for eksempel bare 7-gangen.
- Lag en knapp som bytter mellom gange og pluss.

Husk: Et tilfeldig heltall fra 1 til N er `Math.floor(Math.random() * N) + 1`. Og test etter hvert steg!